Chapter 12 Exercises

DM12-E01

Let G be a finite undirected simple graph with $n \geq 1$ vertices. Prove that G is a tree if and only if G is acyclic and has exactly $n-1$ edges. For the reverse direction, explicitly prove that these conditions imply connectivity; do not assume G is connected.

DM12-E02

A full binary tree has 31 leaves. Find its internal-node and edge counts and the minimum attainable height with root depth zero. Construct a tree attaining this height.

DM12-E03

Use comparison decision trees to prove that sorting six distinct keys needs at least ten comparisons in the worst case. Evaluate $\lceil \log_2 6! \rceil$. Merge-sort [5,1,4,2,6,3], splitting odd lengths with a left half of floor(n/2), and count comparisons by level. If this input uses ten, does that prove optimal worst-case comparison count on all six-key inputs?

DM12-E04

Build a Huffman tree for frequencies 2,3,5,7,11. Give a prefix code, weighted cost, and mean length.

DM12-E05

Prove uniqueness of simple paths in a tree using the cycle formed by two different paths.

DM12-E06

Draw a connected five-vertex graph with a one-child DFS root and a nonroot cut vertex. Apply the low-link cut criterion.

DM12-E07

On edges AB, BC, CA, CD, DE, EF, FD, DG, start alphabetic DFS at A. Trace disc, low, parent, edge-ID pushes, and every complete pop. Find cuts, bridges, and vertex blocks. Then root at D; explain root-child and cut-criterion changes and outputs that must remain invariant.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

DM12-E08

Prove prefix-code lengths (1,2,2,3) impossible, and construct words for (2,2,3,3,3,3). For one symbol, distinguish optimal length zero from a specification requiring nonempty words.

DM12-E09

A full ternary tree has thirteen internal nodes. Find total nodes, leaves, edges, and minimum height with root depth zero. Prove attainability rather than only a lower bound.

DM12-E10

For positive weights $x \le y \le z$, prove the minimum binary-prefix weighted cost and give optimal words. Explain why z at depth two cannot improve the result. Evaluate merge cost and mean length for 2,5,9.

DM12-E11

Prove every tree edge is a bridge and $\sum_v (\deg(v) - 2) = -2$. Find leaves when three vertices have degree three, one degree five, and every other nonleaf degree two. Explain the singleton exception to degree-one leaf counting.

DM12-E12

A loop-free multigraph has vertices u, v and parallel IDs e_1,e_2. DFS discovers v from u along e_1. Use complete low-link rules to decide whether e_1 is a bridge and explain the error from skipping every edge to the parent vertex.

DM12-E13

A graph consists of the two triangles ABC and CDE sharing only C. List its bridges, articulation vertices and vertex blocks under the chapter convention, and draw the block-cut tree. Explain why deleting every bridge does not recover these vertex blocks. Then add one edge BD: prove what changes, using connectivity after vertex deletion rather than relying on the picture.

DM12-E14

Four symbols have positive frequencies a:2, b:5, c:9, d:12. Initially use a:`0`, b:`10`, c:`110`, d:`111`.

```
(a) Keep the shape, swap a,c, then swap b,d. List words, depths, W and its change at each step.
Check whether the minimum-weight symbols are siblings.

(b) Contract a,b to z:7. List reduced words and W, verifying the identity term by term. Is this
reduced tree optimal? Solve the reduced problem and expand z to give optimal original words, W
and mean length.

(c) Compare W and height against fixed two-bit coding. Explain numerically why sibling minima
alone and minimum height alone are insufficient certificates. Use the exchange and contraction
lemmas to establish global optimality, not merely improvement over the initial tree.
```

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 12.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study